Phase 7 Completion Report: PySpark Data Quality Validation

1. Phase Overview

Phase 7 of the Banking Data Governance & Data Quality Pipeline focused on building the PySpark-based data quality validation layer.

The goal of this phase was to practice scalable data profiling and validation using PySpark against the curated banking warehouse datasets. Earlier phases already created the complete pipeline flow from PostgreSQL source data through Python ingestion, raw Parquet files, standardized PySpark outputs, curated fact and dimension models, and SQL-based data quality validation using DuckDB.

Phase 7 added a separate PySpark data quality layer on top of the curated and standardized datasets. This phase did not replace the SQL validation layer from Phase 6. Instead, it added Spark-based profiling, distributed validation, joins, aggregations, reconciliation checks, and exception-record generation.

The Phase 7 validation flow is:

PostgreSQL banking source system
→ Python ingestion layer
→ raw Parquet zone
→ PySpark standardized processing layer
→ standardized Parquet zone
→ curated warehouse fact and dimension layer
→ DuckDB SQL data quality validation layer
→ PySpark data quality validation layer

Phase 7 was completed in two parts:

- Phase 7A: PySpark DQ profiling foundation
- Phase 7B: PySpark DQ rule validation and exception records

2. Phase 7 Objective

The objective of Phase 7 was to build a PySpark-based data quality profiling and validation framework for the banking curated warehouse datasets.

The phase focused on:

- PySpark data profiling
- Schema validation
- Required-column validation
- Null-count profiling
- Distinct-count profiling
- Duplicate logical-key validation

- Accepted-value validation
- Range validation
- Referential-integrity validation using Spark joins
- Source-to-target reconciliation using Spark
- Control-total validation using Spark aggregations
- DQ summary generation
- Bounded exception-record generation
- CSV and Parquet result outputs
- Documentation and README updates

The purpose was to practice data quality checks in a Spark-based environment, where validation logic can scale across larger datasets using DataFrame operations, grouping, joins, and aggregations.

3. Phase 7 Scope

Phase 7 included only the PySpark data quality validation layer.

In Scope

Phase 7 included:

- PySpark DQ configuration
- PySpark DQ validation runner
- Curated dataset profiling
- Column-level profiling
- Schema validation
- Required-column validation
- Duplicate logical-key validation
- Accepted-value checks
- Range checks
- Referential-integrity checks using Spark joins
- Source-to-target row-count reconciliation
- Control-total checks
- Bounded exception-record generation
- CSV and Parquet output files

- Summary CSV logging
- README command updates
- Lightweight documentation

Out of Scope

The following items were intentionally excluded from Phase 7:

- SQL validation script changes
- DuckDB runner changes
- OpenMetadata setup
- Governance YAML artifacts
- Business glossary
- Data catalog YAML
- Lineage YAML
- Agile/JIRA simulation files
- Stewardship automation
- New PySpark transformations
- New curated models
- Dashboarding or reporting tools

This kept Phase 7 focused only on PySpark-based data quality validation.

4. Target Datasets

Phase 7 validated the seven curated warehouse datasets created in Phase 5:

1. dim_customer
2. dim_account
3. dim_branch
4. dim_product
5. fact_transaction
6. fact_customer_risk_score
7. fact_daily_account_balance

Phase 7 also used standardized/source-like datasets where needed for reconciliation and control-total comparisons:

- customers
- accounts
- branches
- products
- transactions
- customer_risk_scores

5. Repository Structure Added or Updated

Phase 7 added and updated the following files:

New or Updated Files

- spark/dq/dq_config.py
- spark/dq/run_pyspark_dq_validation.py
- docs/pyspark_data_quality_validation.md
- README.md

Runtime Output Locations

The following runtime folders were generated locally and ignored by Git:

- data/dq_results/pyspark/load_date=2026-06-16/
- logs/dq/

Runtime data and logs were not committed to the repository.

6. Phase 7A: PySpark DQ Profiling Foundation

Phase 7A Goal

The goal of Phase 7A was to create the foundation for PySpark data quality profiling and basic validation.

Phase 7A focused on building a reusable DQ configuration and runner that could read curated Parquet outputs, profile datasets, validate schemas, validate required columns, check duplicate logical keys, and write result outputs.

Phase 7A Work Completed

Phase 7A created the PySpark DQ foundation under:

- spark/dq/dq_config.py
- spark/dq/run_pyspark_dq_validation.py

The DQ configuration defined the seven curated datasets and included:

- Expected columns
- Required columns
- Logical key columns
- Profile columns
- Dataset names
- Dataset layer information

The runner supported:

- Load-date validation
- Dataset whitelist validation
- Optional selected dataset execution
- Curated Parquet reading
- Dataset-level profiling
- Column-level profiling
- Rule execution
- CSV and Parquet result writing
- Summary CSV generation
- Clear execution logging
- Non-zero failure behavior for execution/configuration errors

Phase 7A Profiling and Validation Categories

Phase 7A implemented the following profiling and validation categories:

1. Dataset profile results:
    - Row count
    - Column count
    - Expected column count
    - Missing expected columns
    - Extra columns
    - Missing required columns
2. Column profile results:
    - Null count

    - Non-null count
    - Distinct count
3. Rule results:
    - Schema expected-columns check
    - Required-columns check
    - Duplicate logical-key check

The duplicate key validation grouped records by each dataset's configured logical key and counted key combinations where the count was greater than one.

Phase 7A Outputs

Phase 7A generated CSV and Parquet result outputs under:

data/dq_results/pyspark/load_date=2026-06-16/

Generated output types:

- dataset_profile_results.csv
- dataset_profile_results.parquet
- column_profile_results.csv
- column_profile_results.parquet
- rule_results.csv
- rule_results.parquet

Phase 7A also generated a timestamped summary CSV under:

logs/dq/

Example:

logs/dq/pyspark_dq_summary_20260618_162312.csv

Phase 7A Validation Results

Phase 7A validation passed successfully.

Validation results:

- Python compilation passed.
- Single dim_customer run succeeded.
- dim_customer run result: 500 rows, 3 rules, 0 failures.
- Full run succeeded for all 7 curated datasets.

- Total rules executed: 21.
- All 21 rules passed.
- Invalid dataset validation failed clearly as expected.
- Runtime outputs were ignored by Git.
- Final Git status was clean.

Phase 7A Commit

Commit:

f0a4abf Phase 7A - Add PySpark DQ profiling foundation

7. Phase 7B: PySpark DQ Rule Validation and Exception Records

Phase 7B Goal

The goal of Phase 7B was to extend the Phase 7A framework with business-rule validation and exception-record generation.

Phase 7B added checks that go beyond basic profiling. It used Spark DataFrame operations, filters, joins, grouping, and aggregations to validate business rules, relationships, reconciliation totals, and control totals.

Phase 7B Work Completed

Phase 7B updated:

- spark/dq/dq_config.py
- spark/dq/run_pyspark_dq_validation.py
- docs/pyspark_data_quality_validation.md
- README.md

Phase 7B added the following DQ categories:

1. Accepted-value validation
2. Range validation
3. Referential-integrity checks using Spark joins
4. Source-to-target row-count reconciliation
5. Control-total validation using Spark aggregations
6. Bounded exception-record generation

Accepted-Value Checks

Phase 7B added accepted-value validation for controlled fields where available.

Examples of accepted-value validations included:

- Customer status values
- Account status values
- Account type values where available
- Transaction type values
- Transaction status values where available
- Risk band or risk rating values where available

These checks validate whether categorical fields contain expected values based on business rules and reference expectations.

Range Checks

Phase 7B added range validation for numeric and financial fields.

Examples of range validations included:

- Transaction amount should be greater than zero.
- Account balance values should remain within configured reasonable ranges.
- Customer risk score should remain within the configured valid scale.
- Daily account balance fields should be present and within configured expected ranges where available.

These checks simulate common financial and risk data validation rules.

Referential-Integrity Checks

Phase 7B added Spark join-based referential-integrity checks between facts and dimensions.

Examples of relationship checks included:

- fact_transaction.account_id to dim_account.account_id
- fact_transaction.customer_id to dim_customer.customer_id
- fact_transaction.branch_id to dim_branch.branch_id where branch_id is not null
- fact_transaction.product_id to dim_product.product_id where product_id is not null
- fact_customer_risk_score.customer_id to dim_customer.customer_id
- fact_daily_account_balance.account_id to dim_account.account_id
- fact_daily_account_balance.customer_id to dim_customer.customer_id

These checks verify that curated fact records do not contain orphaned dimensional references.

Source-to-Target Reconciliation

Phase 7B added Spark-based source-to-target row-count reconciliation between standardized datasets and curated models.

Examples of reconciliation checks included:

- standardized customers to dim_customer
- standardized accounts to dim_account
- standardized branches to dim_branch
- standardized products to dim_product
- standardized transactions to fact_transaction
- standardized customer_risk_scores to fact_customer_risk_score

These checks confirm that curated modeling did not unexpectedly lose records from the standardized source-like layer.

Control-Total Checks

Phase 7B added Spark aggregation-based control-total validation.

Examples of control-total checks included:

- Total transaction amount in standardized transactions compared to fact_transaction
- Transaction counts by transaction type between standardized and curated datasets where fields exist
- Account counts by account status between standardized and curated datasets where fields exist

These checks are important because row-count reconciliation alone is not enough. In banking and financial reporting, amount totals and grouped control totals help confirm that business values remained consistent through transformation.

Exception Records

Phase 7B added bounded exception-record generation.

Exception outputs were designed to capture failed records for data steward or engineer review. Each exception type was bounded to avoid producing large result sets.

Exception records included fields such as:

- Check category
- Check name
- Dataset name
- Business key

- Failed column
- Failed value
- Error message
- Load date

In the Phase 7B validation run, there were zero failed rules and zero exception records. The exception output structure was still generated to support future failed-record review.

Phase 7B Outputs

Phase 7B generated additional CSV and Parquet outputs under:

data/dq_results/pyspark/load_date=2026-06-16/

Output types included:

- Business rule results
- Exception records
- Updated rule result outputs
- Updated summary outputs

Logs were generated under:

logs/dq/

Runtime outputs and logs remained ignored by Git.

Phase 7B Validation Results

Phase 7B validation passed successfully.

Validation results:

- Python compilation passed.
- Single fact_transaction run succeeded.
- Full run succeeded for all 7 curated datasets.
- Accepted-value failed count: 0.
- Range failed count: 0.
- Referential-integrity failed count: 0.
- Reconciliation failed count: 0.
- Control-total failed count: 0.
- Exception record count: 0.

- Runtime outputs remained ignored.
- No SQL or DuckDB files were modified.
- No future-phase files were added.
- Final Git working tree was clean.

Phase 7B Commit

Commit:

59bc5f5 Phase 7B - Add PySpark DQ rule validation and exceptions

8. Final Git History After Phase 7

Final Git log after completing and tagging Phase 7:

59bc5f5 (HEAD -> main, tag: phase-7-complete) Phase 7B - Add PySpark DQ rule validation and exceptions
f0a4abf Phase 7A - Add PySpark DQ profiling foundation
1830481 (tag: phase-6-complete) Phase 6B - Add SQL reconciliation and exception checks
371ca33 Phase 6A - Add DuckDB SQL validation foundation
3fcb13d (tag: phase-5-complete) Phase 5B - Build curated warehouse models

Phase 7 tag:

phase-7-complete

9. Final Phase 7 Status

Phase 7 is complete.

Final status:

Item	Status
Phase 7A PySpark DQ profiling foundation	Complete
Phase 7B PySpark DQ business rule validation	Complete
DQ configuration	Complete
PySpark DQ runner	Complete
Dataset-level profiling	Complete
Column-level profiling	Complete
Schema validation	Complete
Required-column validation	Complete

Item	Status
Duplicate logical-key checks	Complete
Accepted-value checks	Complete
Range checks	Complete
Referential-integrity checks	Complete
Source-to-target reconciliation	Complete
Control-total checks	Complete
Exception-record generation	Complete
CSV result outputs	Complete
Parquet result outputs	Complete
Summary CSV logging	Complete
README updates	Complete
Documentation updates	Complete
Runtime outputs ignored by Git	Complete
Git commits completed	Complete
Phase tag created	Complete

10. Skills Practiced in Phase 7

Phase 7 helped practice the following skills:

- PySpark data quality validation
- PySpark DataFrame operations
- Dataset profiling
- Column profiling
- Schema validation
- Required-column validation
- Null-count profiling
- Distinct-count profiling
- Duplicate key detection

- Accepted-value validation
- Range validation
- Spark join-based referential-integrity checks
- Source-to-target reconciliation
- Control-total validation
- Financial amount validation
- Exception-record generation
- CSV and Parquet output generation
- Runtime summary logging
- Data quality evidence generation
- Configuration-driven validation design
- Command-line Spark job execution
- SDLC-style phased delivery
- Git commit and tagging discipline

11. Governance and Data Quality Relevance

Phase 7 is important because enterprise data governance requires measurable data quality controls, not just documentation.

This phase connects Spark-based engineering work to governance practice in the following ways:

- Critical datasets were profiled.
- Required fields were validated.
- Logical keys were checked for duplicates.
- Controlled fields were checked for accepted values.
- Numeric and financial fields were validated using range checks.
- Fact-to-dimension relationships were validated through Spark joins.
- Source-to-target record movement was reconciled.
- Financial control totals were compared.
- Failed-record exception outputs were created for future stewardship review.
- Results were written in repeatable CSV and Parquet formats.
- Summary logs provided validation evidence.

This is similar to the type of data quality validation that supports governed data platforms, data steward review, audit readiness, regulatory reporting, and trust in curated warehouse datasets.

12. Difference Between Phase 6 and Phase 7

Phase 6 and Phase 7 both validate data quality, but they practice different skills.

Phase 6 focused on SQL-based validation using DuckDB over Parquet files. It used SQL scripts to validate nulls, duplicates, accepted values, referential integrity, reconciliation, control totals, and exception records.

Phase 7 focused on PySpark-based validation. It used Spark DataFrame logic, grouping, joins, filters, and aggregations to perform scalable profiling and validation.

The key distinction is:

- Phase 6 practiced SQL data quality validation.
- Phase 7 practiced PySpark data quality validation.

This separation was intentional so that both SQL and PySpark data quality skills could be practiced clearly without mixing implementation styles.

13. Interview Explanation

A clear interview explanation for Phase 7 would be:

“In Phase 7, I built the PySpark-based data quality validation layer for my banking governance pipeline. Earlier phases had already created the source data, raw Parquet zone, standardized PySpark layer, curated warehouse models, and SQL validation layer. In Phase 7, I added a separate Spark-based DQ framework so I could practice scalable profiling and validation using PySpark.

I split the work into two parts. In Phase 7A, I created a configuration-driven PySpark DQ runner that reads curated Parquet datasets, profiles row counts and column counts, calculates null and distinct counts by column, validates expected and required columns, and checks duplicate logical keys. The runner writes dataset profile results, column profile results, and rule results in both CSV and Parquet formats.

In Phase 7B, I extended the framework with business-rule validation. I added accepted-value checks, range checks, referential-integrity checks using Spark joins, source-to-target reconciliation, control-total checks using Spark aggregations, and bounded exception-record outputs. The full run completed successfully across all seven curated datasets, with zero failed rule counts and zero exception records.

This phase gave me hands-on practice with PySpark data profiling, data quality rule execution, Spark joins, aggregations, reconciliation, exception handling, and validation evidence generation in a governance-focused banking data pipeline.”

14. Phase 7 Conclusion

Phase 7 successfully added the PySpark Data Quality Validation layer to the Banking Data Governance & Data Quality Pipeline.

The project now has a complete flow from source generation through both SQL and PySpark validation:

PostgreSQL banking source system
→ Python ingestion layer
→ raw Parquet zone
→ PySpark standardized processing layer
→ standardized Parquet zone
→ curated warehouse fact and dimension layer
→ DuckDB SQL data quality validation layer
→ PySpark data quality validation layer

Phase 7 confirmed that all seven curated datasets passed the implemented PySpark profiling and validation checks.

Final Phase 7 result:

- Phase 7A completed and committed.
- Phase 7B completed and committed.
- Phase 7 tagged as phase-7-complete.
- All seven curated datasets validated successfully.
- All failed-rule counts were zero.
- Exception record count was zero.
- Runtime files were ignored by Git.
- Final Git status was clean.

The project is now ready for Phase 8: Governance Artifacts.